Отзыв руководителя

Содержание

Введение 6
1 Обоснование выбора темы курсового проекта 7
1.1 Анализ предметной области 7
1.2 Аналогичные программные решения 8
1.3 Требования к программному продукту 11
1.4 Обоснование выбора инструментальных средств разработки 12
2 Разработка приложения 14
2.1 Разработка блок-схем программы 14
2.2 Макеты основных страниц приложения 22
2.3 Разработка пользовательского интерфейса 26
2.4 Инструкция по работе с продуктом 31
2.5 Разработка API 36
Заключение 38
Библиография 39
Приложение А 40

1 Вариант задания демоэкзамена

1.1 Краткая спецификация разрабатываемого модуля

Модуль учета заявок на ремонт автомобилей для автосервиса "АвтоТранс".
Назначение: автоматизация учета и обработки заявок.
Входные данные:
- данные новой заявки (ФИО, телефон, автомобиль, описание проблемы);
- данные для авторизации (логин, пароль);
- изменение статусов и комментариев от механика;
- параметры для поиска (номер, ФИО заказчика).

Выходные данные:
- полный список заявок;
- статистические показатели;
- информационные сообщения;
- результаты поиска.

Основной алгоритм решения учёта заявок на ремонт автомобилей в виде блок-схемы (Рисунок 1.1).

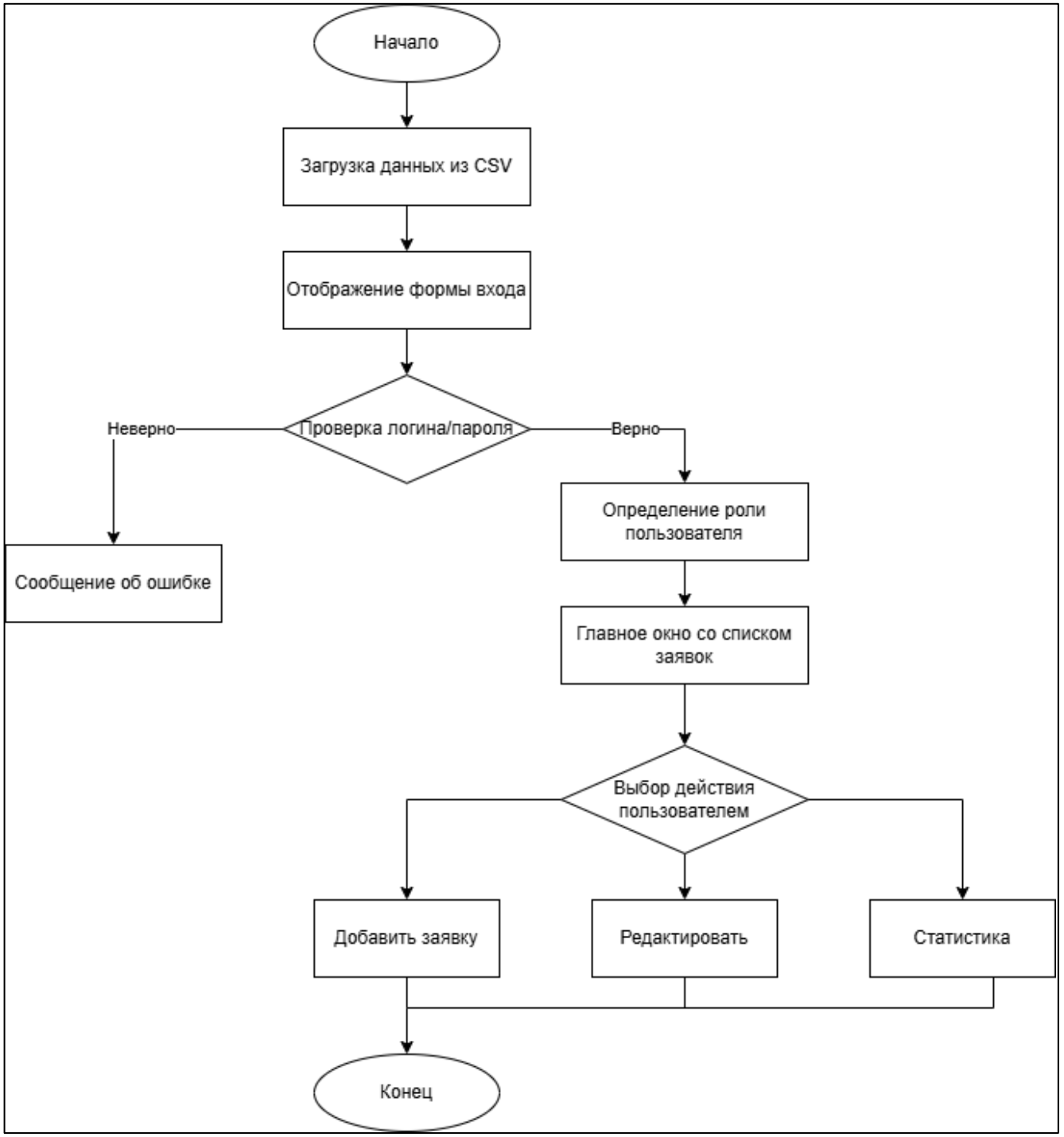

Рисунок 1.1 – Блок-схема основного алгоритма

Блок-схема функции расчёта количества заявок (Рисунок 1.2).

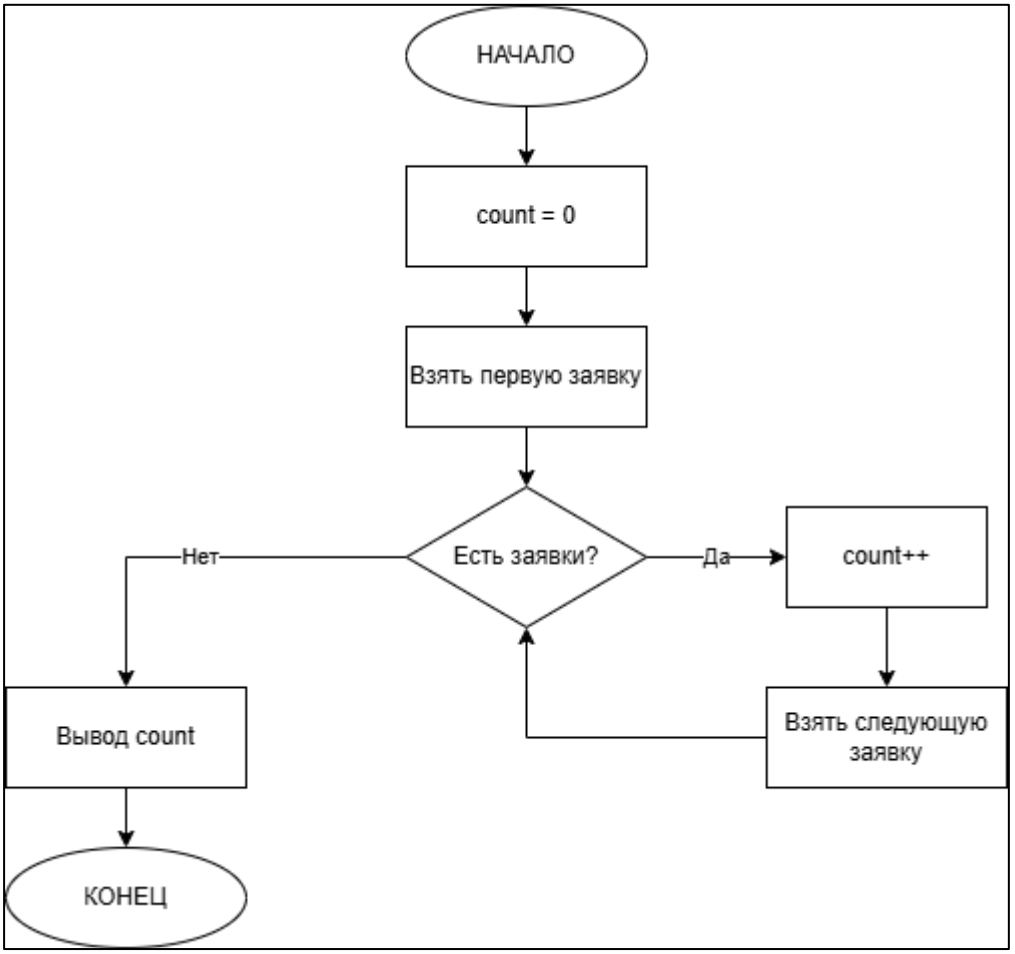

Рисунок 1.2 - Блок-схема функции расчёта количества заявок

ER-диаграмма базы данных (Рисунок 1.3).

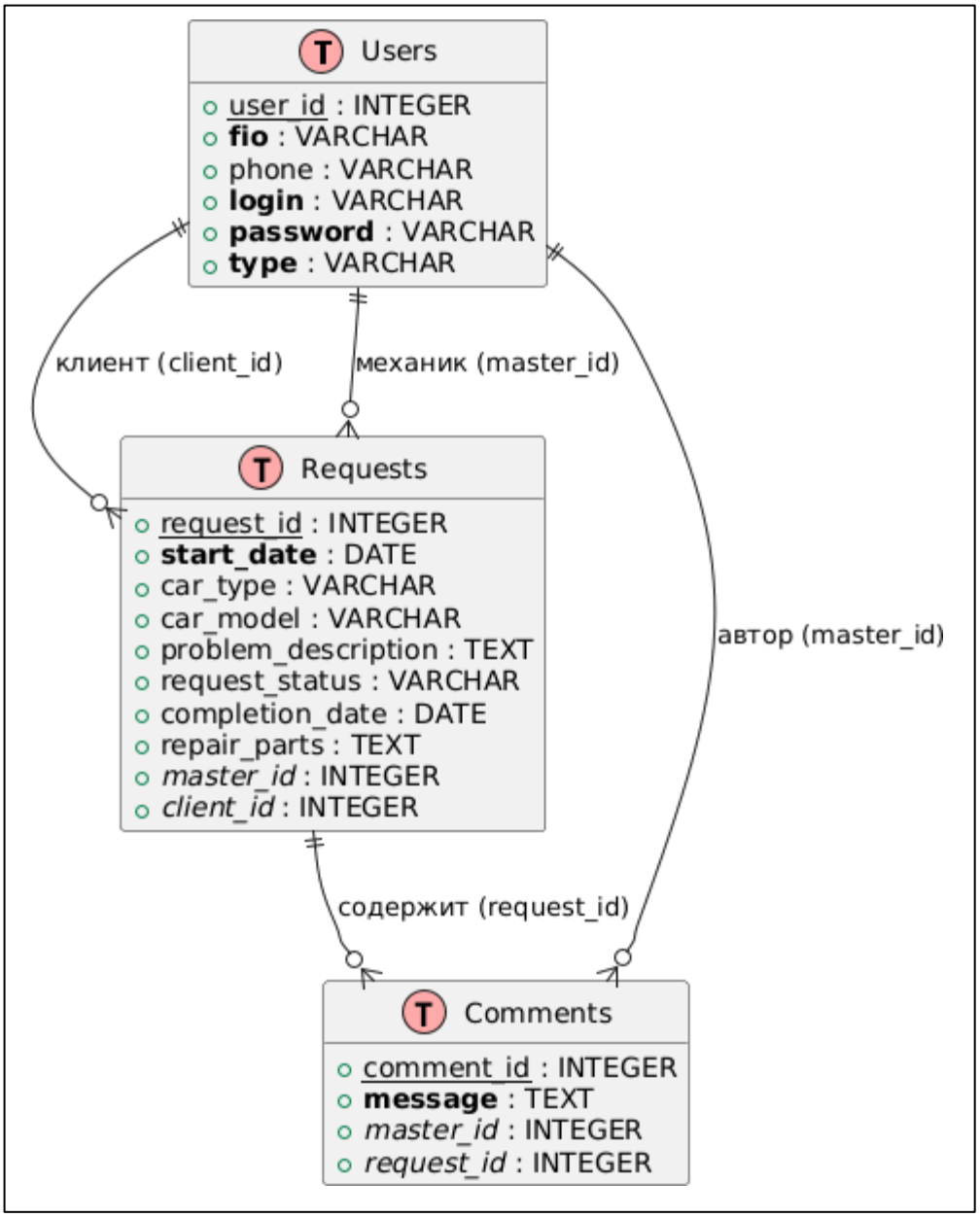

Рисунок 1.3 – ER-диаграмма

1.2 Функциональные и нефункциональные требования

Функциональные требования:

- возможность добавления заявок в бд с указанием параметров: номер заявки, дата добавления, вид авто, модель авто, описание проблемы, ФИО клиента, номер телефона, статус заявки;
- возможность редактирования заявок (этап выполнения, описание проблемы, ответственный за выполнение работ);
- просмотр списка заявок;
- поиск по номеру заявки;
- поиск по параметрам;
- количество выполненных заявок;
- среднее время выполнения заявки;
- статистика по типам неисправностей;
- добавление автомеханика к заявке;
- отслеживание состояния работы;
- добавление комментариев к заявке автомехаником (автозапчасти, материалы, т.п.).

Нефункциональные требования:

- поддержка windows;
- простой интерфейс, понятные уведомления;
- быстрота работы;
- система авторизации и роли.

1.2 Обоснование выбора инструментальных средств разработки

Выбор языка программирования: C# и платформа .NET.

Преимущества языка программирования C# и платформы .NET:

1. Надежность и типобезопасность. Строгая типизация C# позволяет выявлять многие ошибки ещё на этапе компиляции, что критически важно для бизнес-приложений, работающих с большим объёмом данных о заявках, клиентах и ремонтах. Это снижает вероятность сбоев во время выполнения и повышает общую стабильность системы.
2. Высокая производительность. Платформа .NET Core (современные версии .NET 6/7/8) обеспечивает производительность, сопоставимую с нативными языками (C++, Go). Это гарантирует быстрый отклик интерфейса даже при одновременной работе нескольких пользователей (операторов, механиков, менеджеров) и росте количества заявок.
3. Экосистема и библиотеки. .NET обладает одной из самых богатых экосистем. Для реализации вашего проекта есть готовые, проверенные решения для:

4. Кроссплатформенность. Приложение на .NET может быть развёрнуто не только на Windows, но и на Linux-серверах, что даёт гибкость при выборе хостинга и снижает эксплуатационные расходы.

5. Безопасность. Платформа предоставляет встроенные механизмы защиты от распространённых угроз: межсайтового скриптинга (XSS), подделки межсайтовых запросов (CSRF) и SQL-инъекций (за счёт использования параметризованных запросов в EF Core).

Выбор среды разработки: Visual Studio 2022.

Преимущества среды разработки Visual Studio 2022:

1. Интегрированная среда разработки (IDE) "все в одном". Visual Studio — это мощнейшая IDE, специально созданная для разработки на .NET. Она предоставляет все необходимые инструменты из коробки:

2. Удобство работы с базами данных. Visual Studio имеет встроенную поддержку подключения к PostgreSQL (через расширения или прямые инструменты). Можно выполнять SQL-запросы, просматривать структуру таблиц и управлять данными непосредственно из среды. Кроме того, интеграция с Entity Framework Core позволяет легко создавать модели на основе существующей базы (Database First) или генерировать миграции для синхронизации схемы.

3. Повышение производительности разработчика. Функции IntelliSense, автодополнение кода, быстрая навигация по проекту и рефакторинг значительно ускоряют написание кода на C# и Razor. Это особенно важно при внесении изменений в контроллеры и представления.

4. Поддержка современных стандартов. Visual Studio 2022 полностью поддерживает последние версии .NET, C# и инструменты тестирования, что гарантирует актуальность разрабатываемого решения.

1.3 Разработка интерфейса программного модуля

При регистрации автоматически создаётся роль «заказчик».
Для входа как «менеджер»: логин: login1, пароль: pass1.
Для входа как «механик»: логин: login2, пароль: pass2.
Для входа как «заказчик»: логин: login12, пароль: pass12.

Окно авторизации представлено на рисунке 1.4. Имеются поля для ввода логина и пароля, кнопка «Войти». Для пользователей, не имеющих авторизованный профиль, имеется ссылка на окно регистрации.

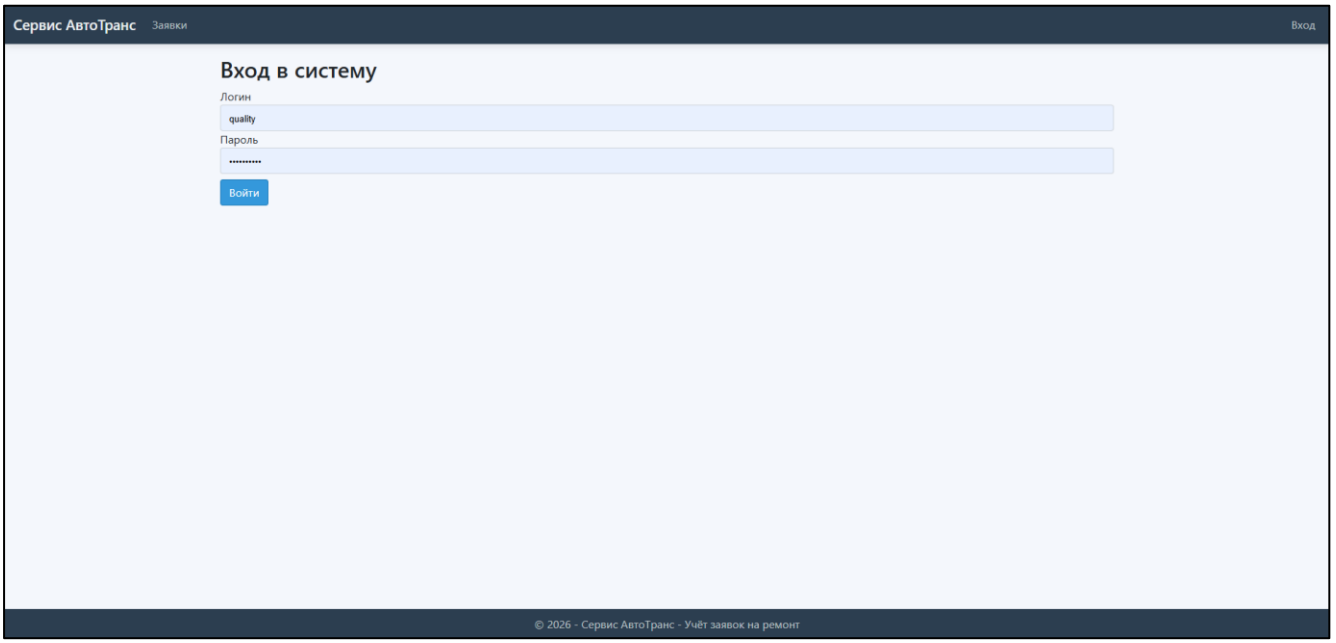

Рисунок 1.4 – Окно авторизации

Окно регистрации представлено на рисунке 1.5. Имеются поля для ввода ФИО, номера телефона, логина, пароля, а также для подтверждения пароля, кнопка «Зарегистрироваться». Для пользователей, имеющих авторизованный профиль, имеется ссылка на окно входа.

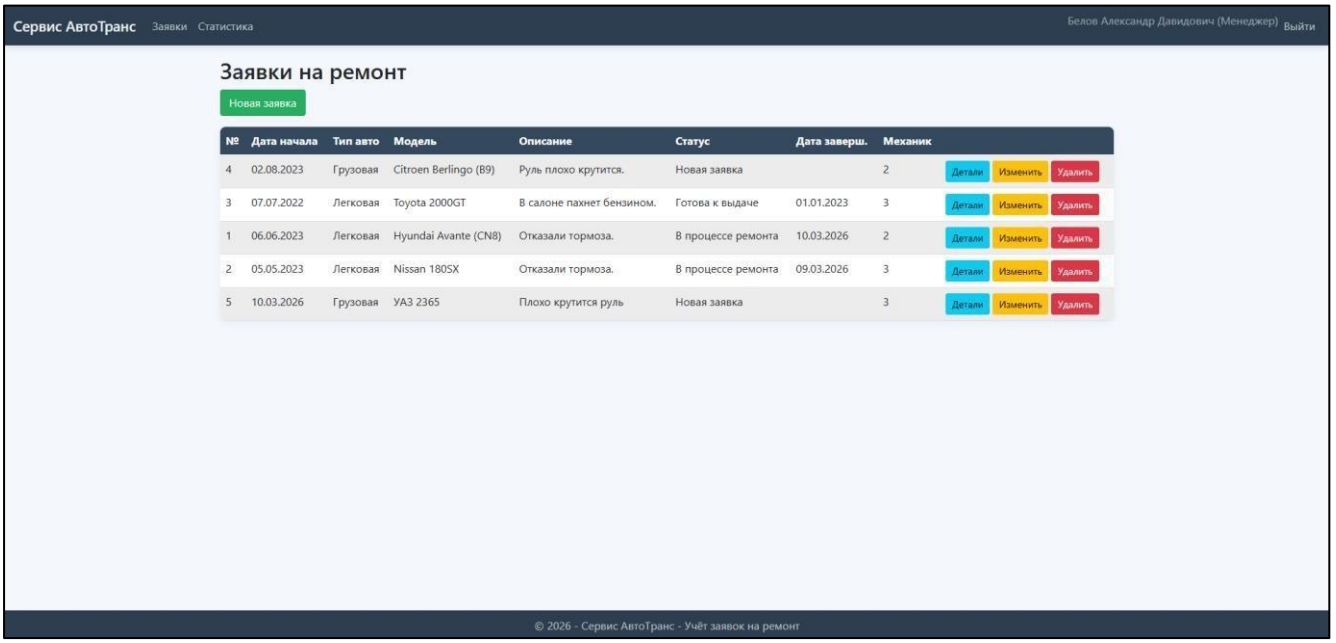

Рисунок 1.5 – Главная страница

На рисунке 1.6 представлено окно просмотра заявки (для пользователя «менеджер»). В этом окне есть возможность просмотра всех деталей заявки. Имеется поле для оставления комментария, кнопка «Редактировать заявку».

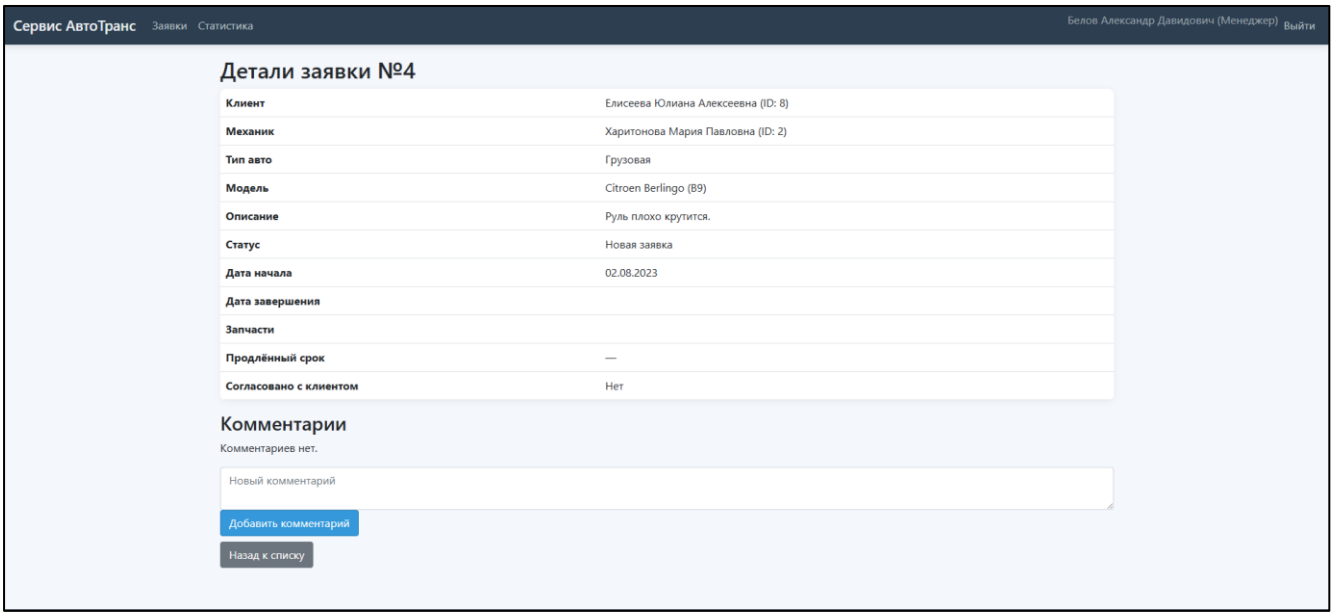

Рисунок 1.6 – Окно просмотра

Окно редактирования заявки представлено на рисунке 1.7 (для пользователя «менеджер»). Для удобства пользователя имеются выпадающие списки для выбора вида автомобиля (Рисунок 1.8), клиента (Рисунок 1.9), статуса заявки (Рисунок 1.10) и механика (Рисунок 1.11).

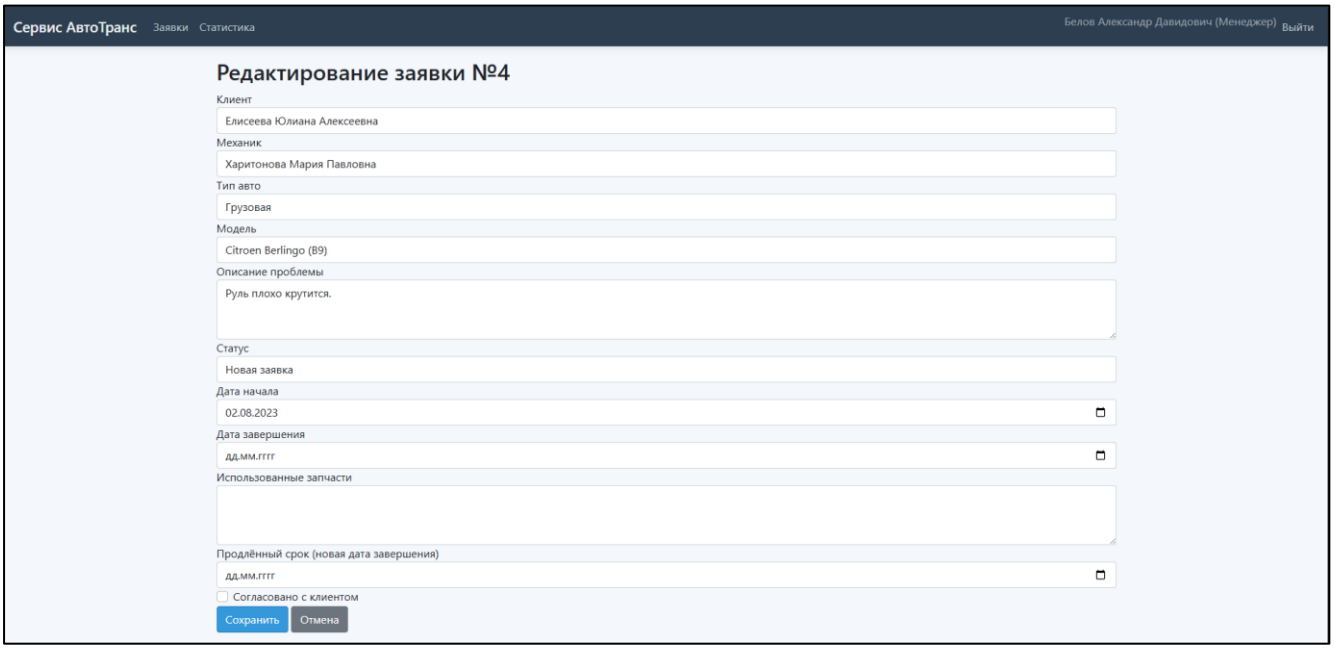

Рисунок 1.7 – Окно редактирования

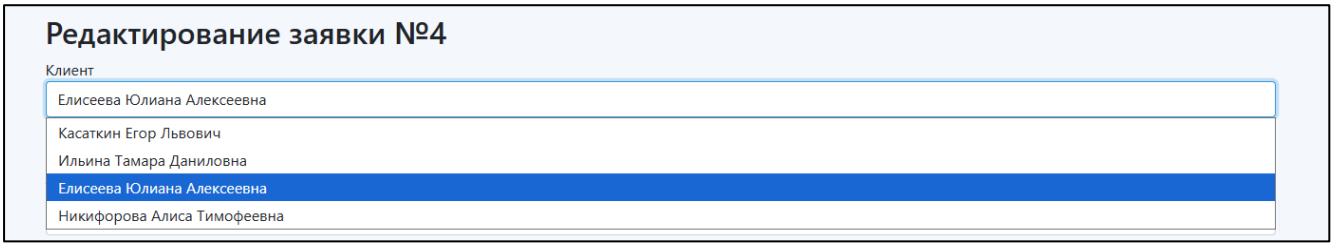

Рисунок 1.8 – Выпадающий список

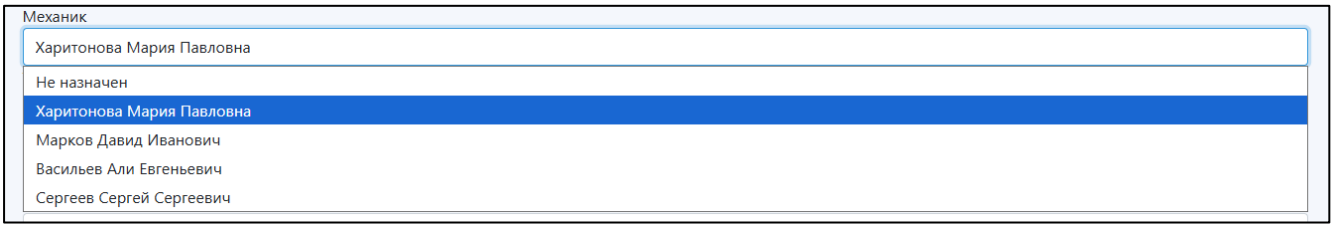

Рисунок 1.9 – Выпадающий список

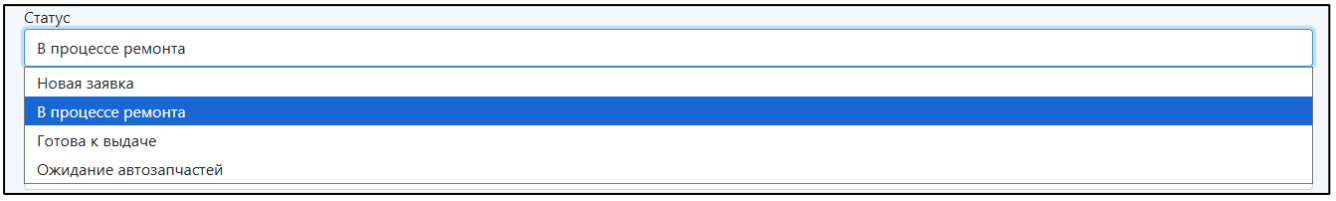

Рисунок 1.10 – Выпадающий список

На рисунке 1.11 представлено окно добавления новой заявки (для пользователя «менеджер»). Имеются поля ввода модели автомобиля и описания проблемы, выпадающие списки для выбора вида автомобиля и клиента.

Рисунок 1.11 – Окно добавления новой заявки

На рисунке 1.12 представлена главная страница с заявками для пользователя «заказчик». Имеется возможность просмотра собственных заявок, поиска по параметрам, просмотра и редактирования.

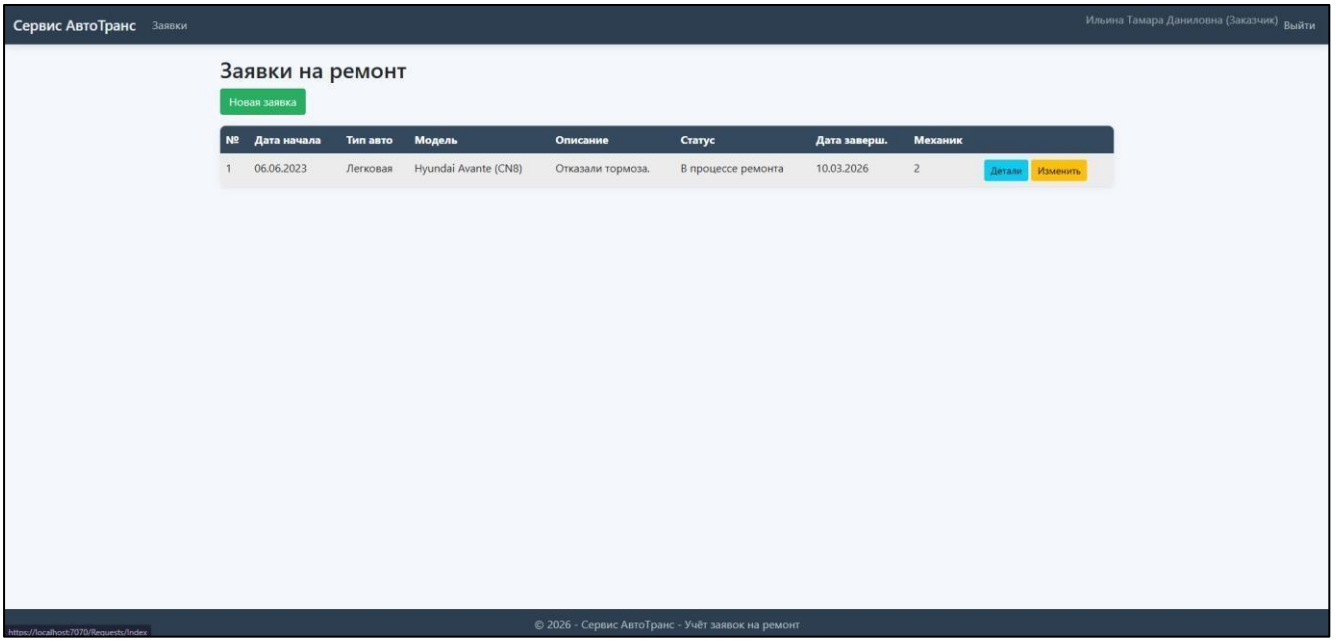

Рисунок 1.12 – Главная страница

Окно добавления новой заявки представлено на рисунке 1.13 (для пользователя «заказчик»). Имеются поля ввода модели автомобиля и описания проблемы, выпадающий список для выбора вида автомобиля.

Рисунок 1.13 – Окно добавления заявки

На рисунке 1.14 представлено окно просмотра заявки (для пользователя «заказчик»). Имеется возможность просмотра всех деталей заявки.

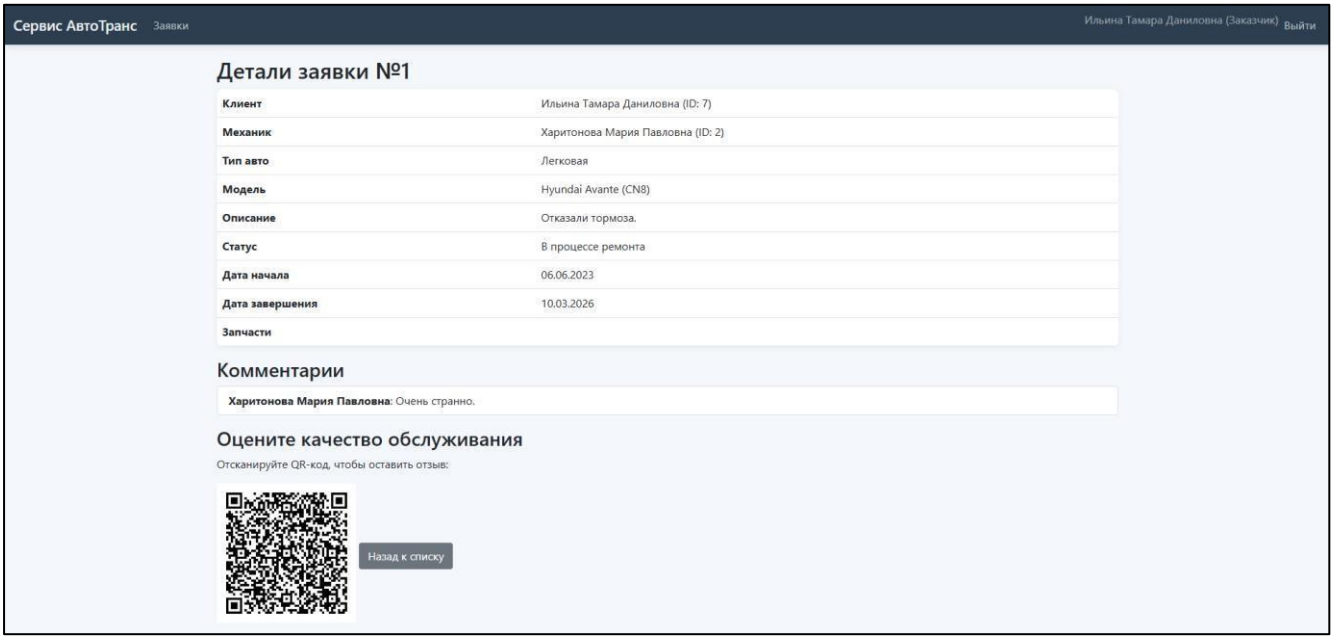

Рисунок 1.14 – Окно просмотра заявки

На рисунке 1.15 представлено окно редактирования заявки (для пользователя «заказчик»). Имеются поля для ввода модели автомобиля и описания проблемы, выпадающий список для выбора вида автомобиля.

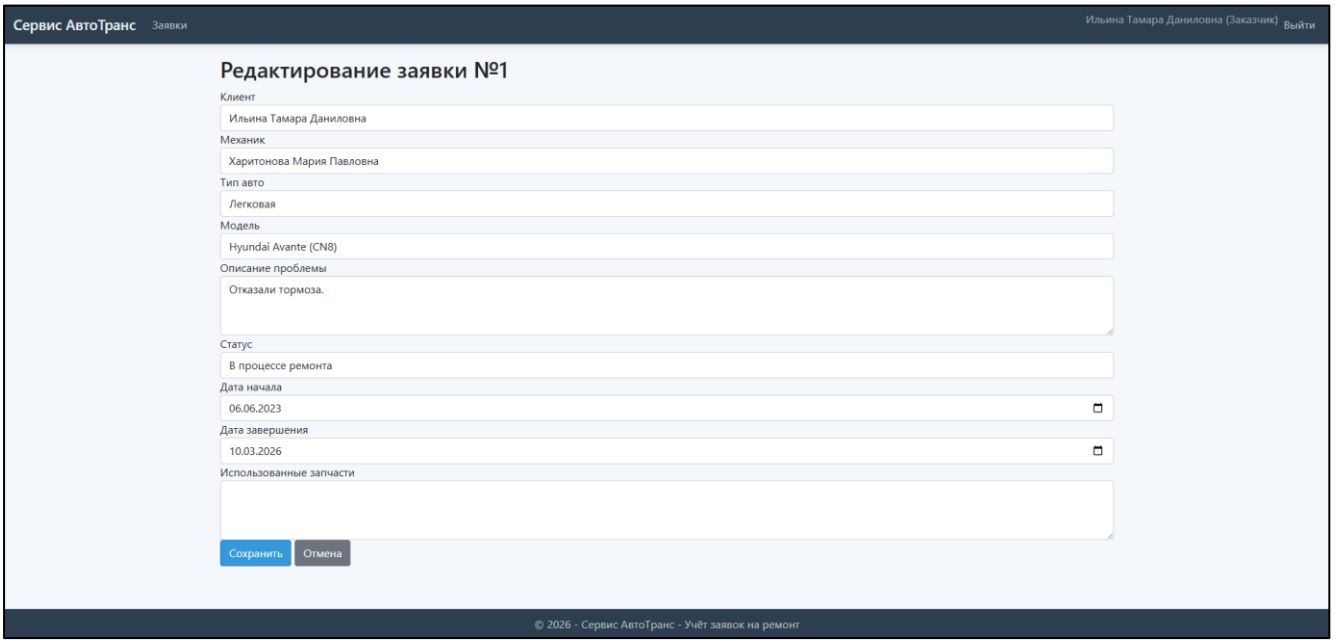

Рисунок 1.15 – Окно редактирования заявки

На рисунке 1.16 представлена главная страница с заявками для пользователя «механик». Имеется возможность просмотра собственных заявок, поиска по параметрам, просмотра и редактирования.

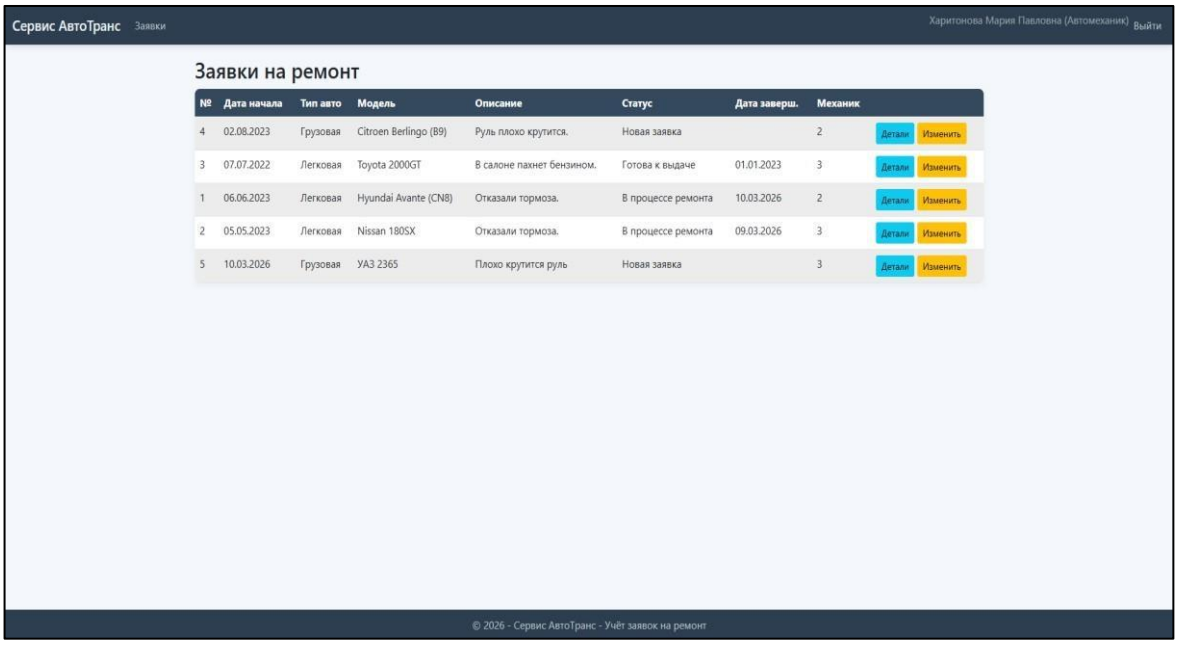

Рисунок 1.16 – Главная страница

На рисунке 1.17 представлено окно просмотра заявки (для пользователя «механик»). Имеется возможность просмотра всех деталей заявки, оставления комментариев и кнопка «Редактировать заявку».

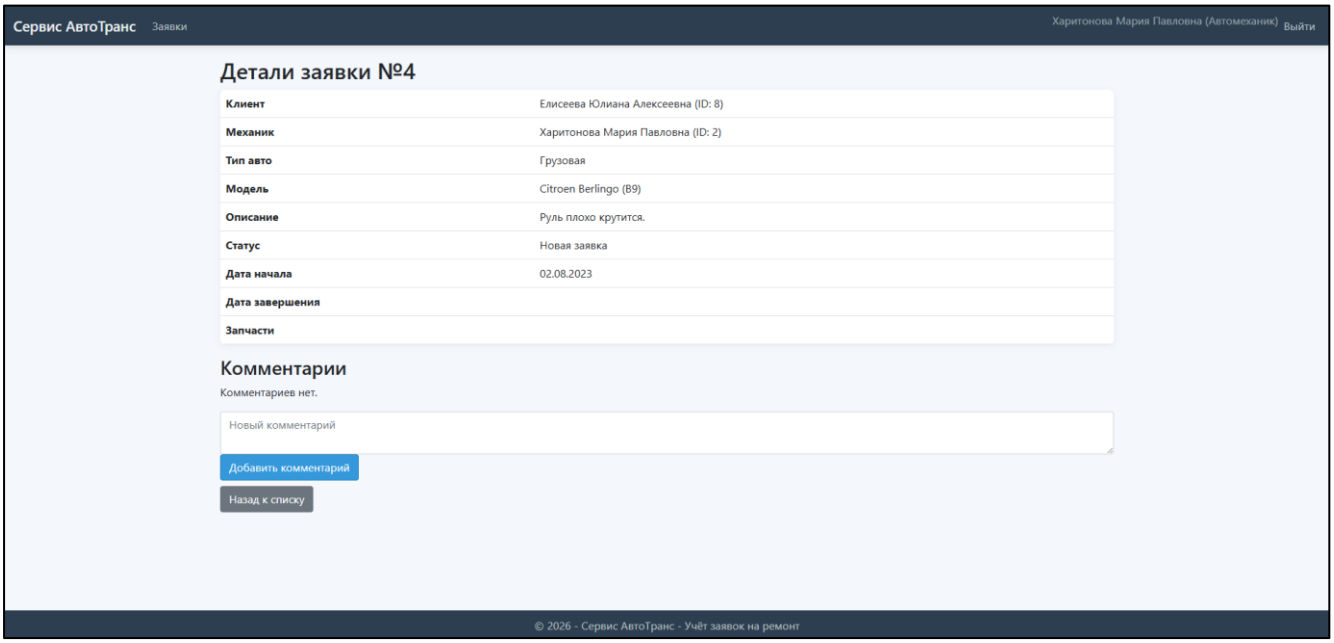

Рисунок 1.17 – Окно просмотра заявки

На рисунке 1.18 представлено окно редактирование заявки (для пользователя «механик»). Имеется выпадающий список для выбора статуса заявки и поле для ввода запчастей/материалов.

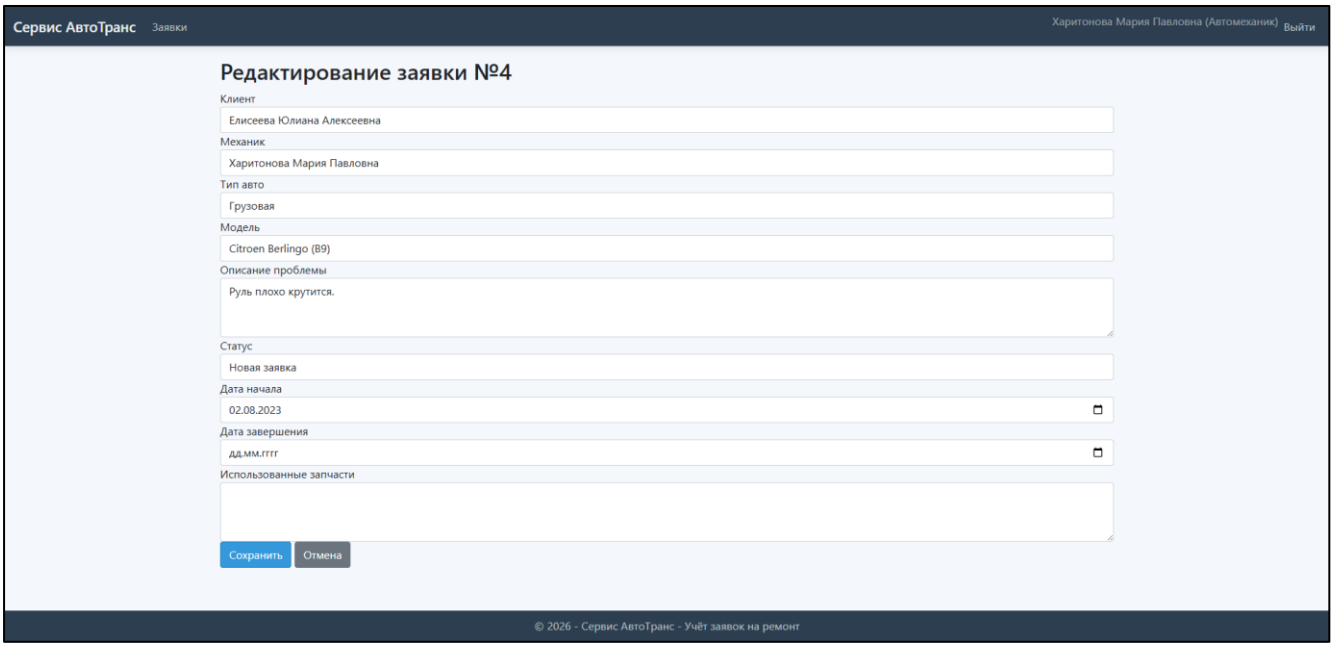

Рисунок 1.18 – Окно редактирования заявки

2 Описание практической части дипломного проекта

2.2 Обоснование выбора инструментальных средств разработки

Для разработки приложения использовался следующий стек: Серверная часть:

− Node.js v24.11.0 - серверная платформа на JavaScript, обеспечивающая высокую производительность и м;асштабируемость за счет асинхронной обработки запросов;

− Express.js 4.18.2 - минималистичный веб-фреймворк для создания RESTful API;

− PostgreSQL 18 - реляционная система управления базами данных для хранения метрик KPI, пользовательских данных и конфигураций; − pg 8.11.3 - нативный драйвер PostgreSQL для Node.js.

Клиентская часть:

− React 18.3.1 - библиотека для построения пользовательского интерфейса с компонентным подходом;

− TypeScript 5.5.3 - язык программирования для добавления статической типизации, повышающей надежность кода;

− Vite 5.4.2 - современный сборщик и инструмент для разработки с горячей перезагрузкой модулей;

− Tailwind CSS 3.4.1 - утилитарный CSS-фреймворк для быстрой и удобной стилизации интерфейса;

− Chart.js - библиотека для создания интерактивных диаграмм и графиков визуализации KPI.

Вспомогательные технологии:

− bcryptjs 2.4.3 - библиотека для безопасного хеширования паролей пользователей;

− jsonwebtoken 9.0.2 - реализация JSON Web Tokens для аутентификации и авторизации;

− multer 2.0.2 - middleware для обработки загрузки файлов (CSV, Excel);

− dotenv 16.3.1 – управление переменными окружения для конфигурации приложения.

В случае с backend-частью приложения, то выбор Node.js и Express.js обусловлен их высокой производительностью при обработке большого количества одновременных запросов, что критично для системы мониторинга KPI в реальном времени. Асинхронная модель Node.js позволяет эффективно обрабатывать вычисления метрик и работу с базой данных без блокировки основного потока выполнения.

Frontend было решено писать на React с TypeScript и Vite, так как React позволяет создавать переиспользуемые компоненты для виджетов дашборда, является реактивным и обеспечивает высокую производительность при частых обновлениях данных KPI. TypeScript добавляет статическую типизацию, что повышает надежность кода и снижает количество ошибок при работе со сложной бизнес-логикой расчета метрик. Vite обеспечивает быструю сборку и удобную разработку с горячей перезагрузкой. В сочетании с Tailwind CSS это позволяет быстро создавать адаптивные и интуитивно понятные интерфейсы, соответствующие требованиям корпоративных пользователей. React также поддерживает множество библиотек для визуализации данных (таких как Chart.js) и содержит готовые решения для маршрутизации (React Router), что ускоряет разработку и знакомо разработчикам.

PostgreSQL как СУБД и pgAdmin 4 в качестве инструмента администрирования были указаны в требованиях проекта.

Выбранный стек технологий представляет собой единое интегрированное решение, которое обеспечивает следующие ключевые преимущества:

− единая языковая экосистема и полная типобезопасность - использование JavaScript/TypeScript на всех уровнях приложения (фронтенд, бэкенд, БД) снижает сложность разработки, обеспечивает сквозную типизацию данных и позволяет повторно использовать код, что ускоряет создание и поддержку системы;

− высокая производительность и отзывчивость интерфейса - React с виртуальным DOM обеспечивает мгновенные обновления данных KPI, а асинхронная архитектура Node.js эффективно обрабатывает одновременные запросы множества пользователей, что критично для мониторинга в реальном времени;

− оптимизированный цикл разработки и современный инструментарий - Vite ускоряет сборку и обеспечивает горячую перезагрузку, Tailwind CSS позволяет быстро создавать адаптивные интерфейсы, а Chart.js предоставляет готовые решения для интерактивной визуализации данных;

− надежное хранение данных и промышленная стабильность - PostgreSQL гарантирует целостность и сохранность исторических данных KPI, поддерживает сложные аналитические запросы, а pgAdmin 4 предоставляет удобный инструмент для администрирования базы данных.

Единое приложение обеспечивает бесшовное взаимодействие всех компонентов стека, создавая высокопроизводительную, типобезопасную и легко масштабируемую платформу для мониторинга KPI, где фронтенд и бэкенд оптимально дополняют друг друга в единой экосистеме JavaScript/TypeScript.

2.3 Разработка пользовательского интерфейса

Пользовательский интерфейс должен быть интуитивно понятен пользователю и удобен в использовании. Он должен позволять пользователю взаимодействовать с системой для управления метриками KPI, создания и настройки дашбордов путём взаимодействия с REST API. Backend представляет собой API-сервер для обмена данными между базой данных, в которой хранятся показатели эффективности, и веб-интерфейсом, через определённые на сервере эндпоинты.

Список показателей KPI должен отображаться в виде таблицы. Для каждого показателя в таблице должно отображаться:

- наименование KPI;
- текущий статус (активен, неактивен);
- отдел или категория, к которой относится показатель;
- дата и время последнего расчёта.

Должна быть предусмотрена возможность работы с CRUD операциями.

Присутствуют страницы для различных действий с данными, например, страница с заказами, сотрудниками и информацией о профиле.

В интерфейсе должен быть раздел «История показателей» или «Архив KPI». Данные должны отображаться в таблице с возможностью фильтрации и поиска по дате, отделу, сотруднику, категории показателя. Пользователь должен иметь возможность загружать исходные данные из файлов (CSV, Excel) для использования в расчётах. Пользователь должен иметь возможность через веб-интерфейс изменять параметры расчёта KPI, формулы, целевые значения и настройки визуализации.

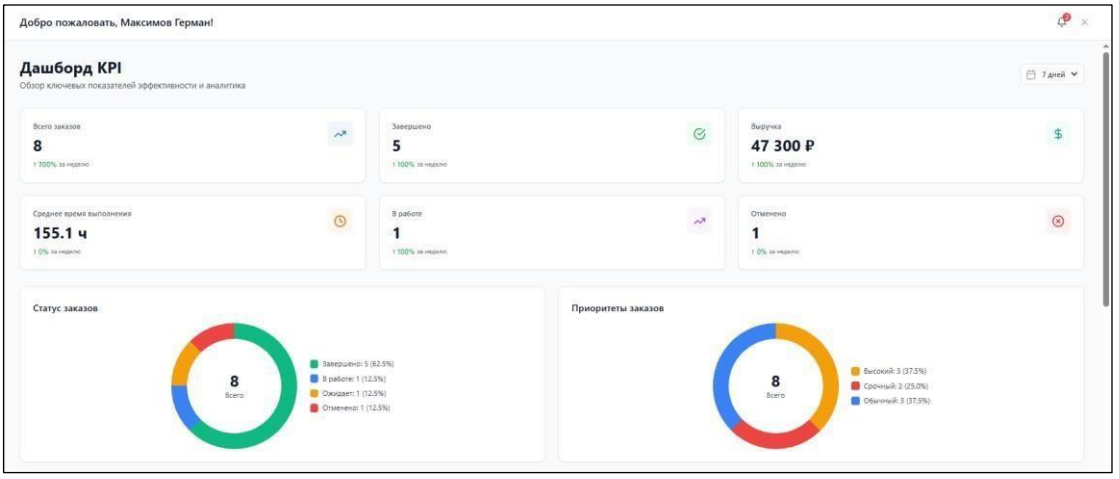

Рисунок 2.11 – Главная страница дашборда

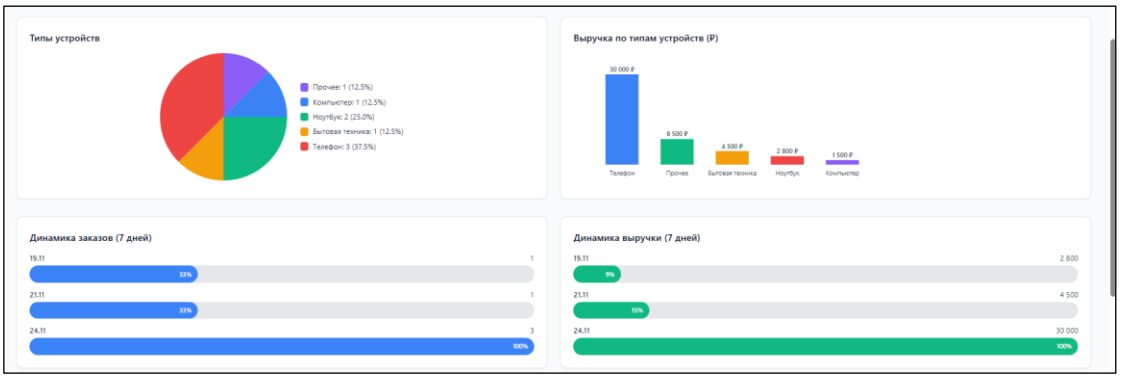

Рисунок 2.12 – Продолжение главной страницы дашборда

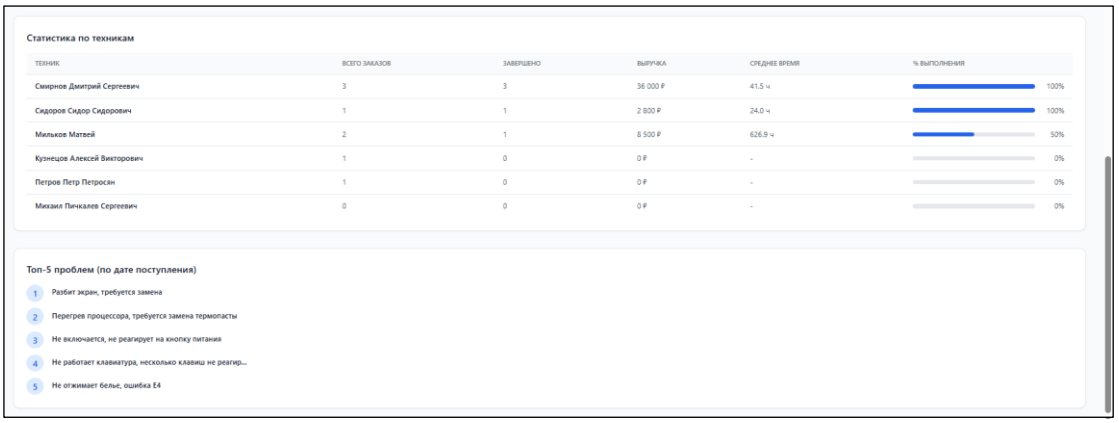

Рисунок 2.13 – Главная страница дашборда

На странице, наблюдаемой на рисунках 2.11-2.13 можно видеть все диаграммы, графики и динамики, статистику по техникам и самые популярные проблемы.

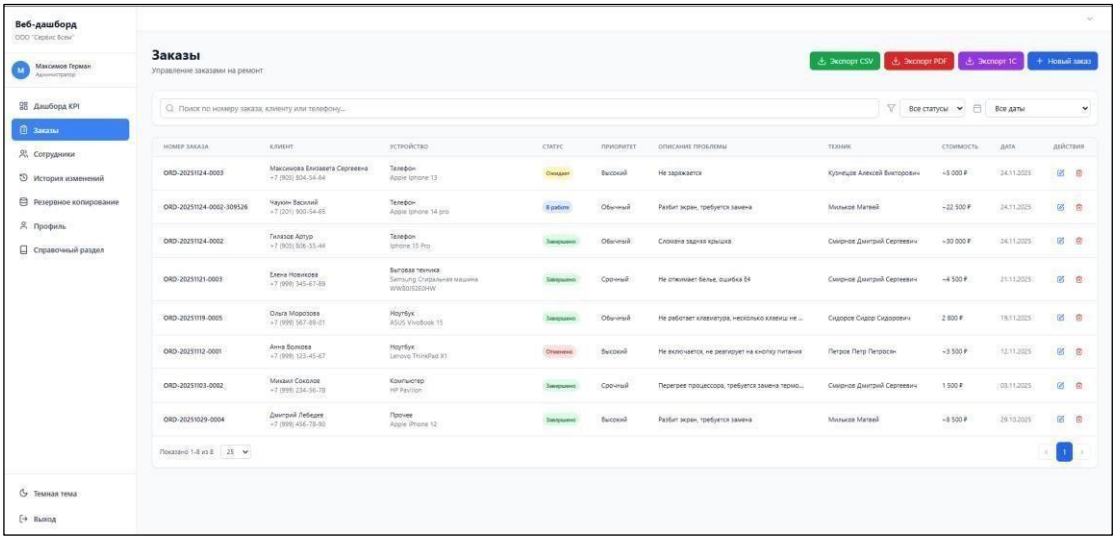

Рисунок 2.14 – Страница заказы

На странице, наблюдаемой на рисунке 2.14 можно видеть все активные заказы, также можем их добавлять, удалять и редактировать, а также экспортировать в PDF, CSV, 1C. Для удобства можно применить фильтрацию и пагинацию.

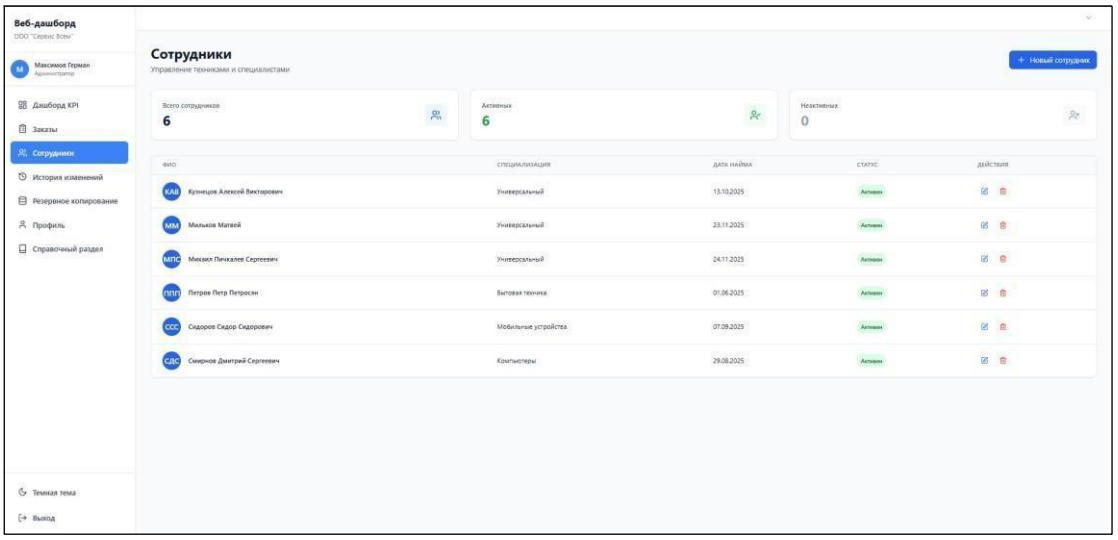

Рисунок 2.15 – Страница сотрудники

На странице, наблюдаемой на рисунке 2.15 можно просматривать, нанимать, удалять (переводить в отпуск) и редактировать информацию сотрудников.

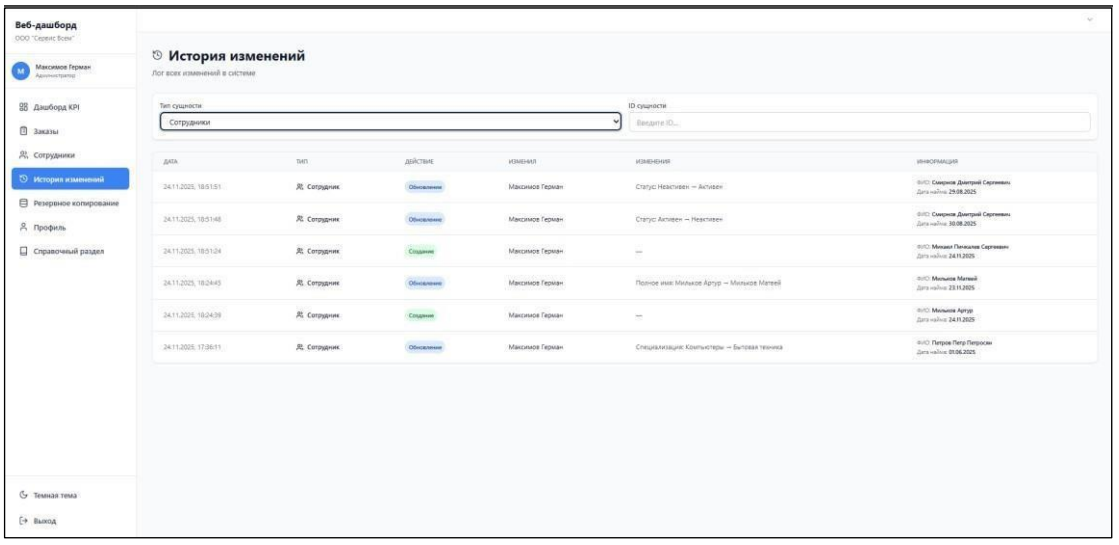

Рисунок 2.16 – Страница аудита (истории изменений)

На странице, наблюдаемой на рисунке 2.16 можно увидеть аудит (историю изменений) где все изменения в любой из страниц будут логироваться.

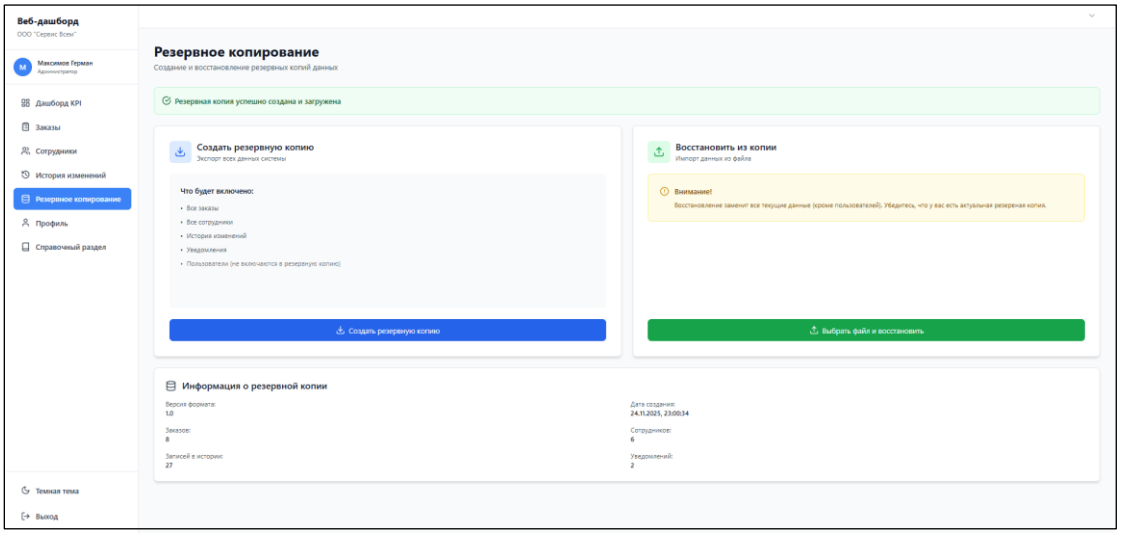

Рисунок 2.17 – Страница резервного копирования

На странице, наблюдаемой на рисунке 2.17 реализована функция резервного копирования, а также восстановления из неё.

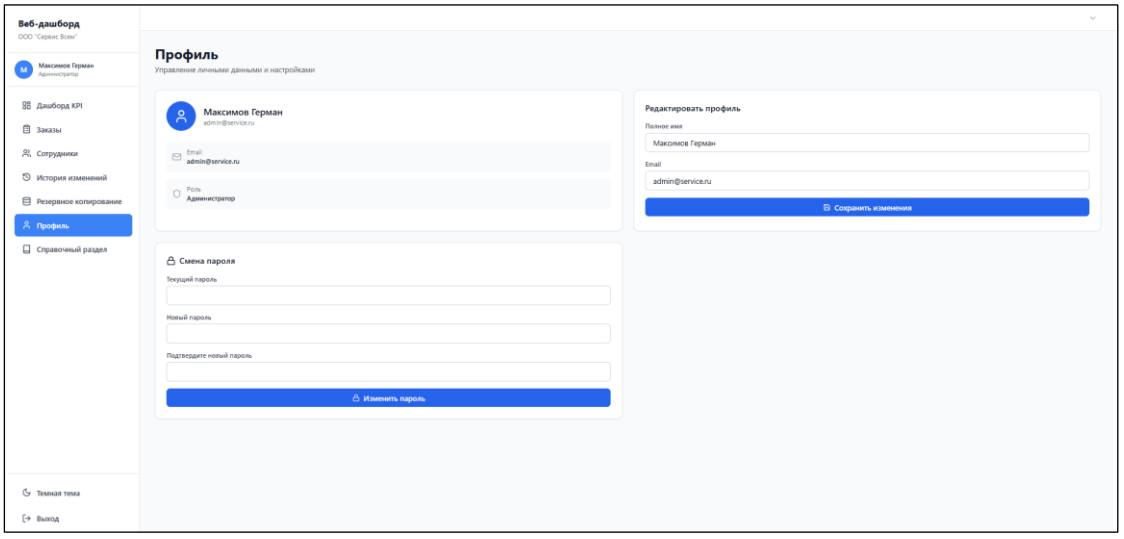

Рисунок 2.18 – Страница профиль

На странице, наблюдаемой на рисунке 2.18 можно наблюдать информацию профиля, а именно имя пользователя, роль и электронная почта (логин), а также её можно поменять, как и пароль от учетной записи.

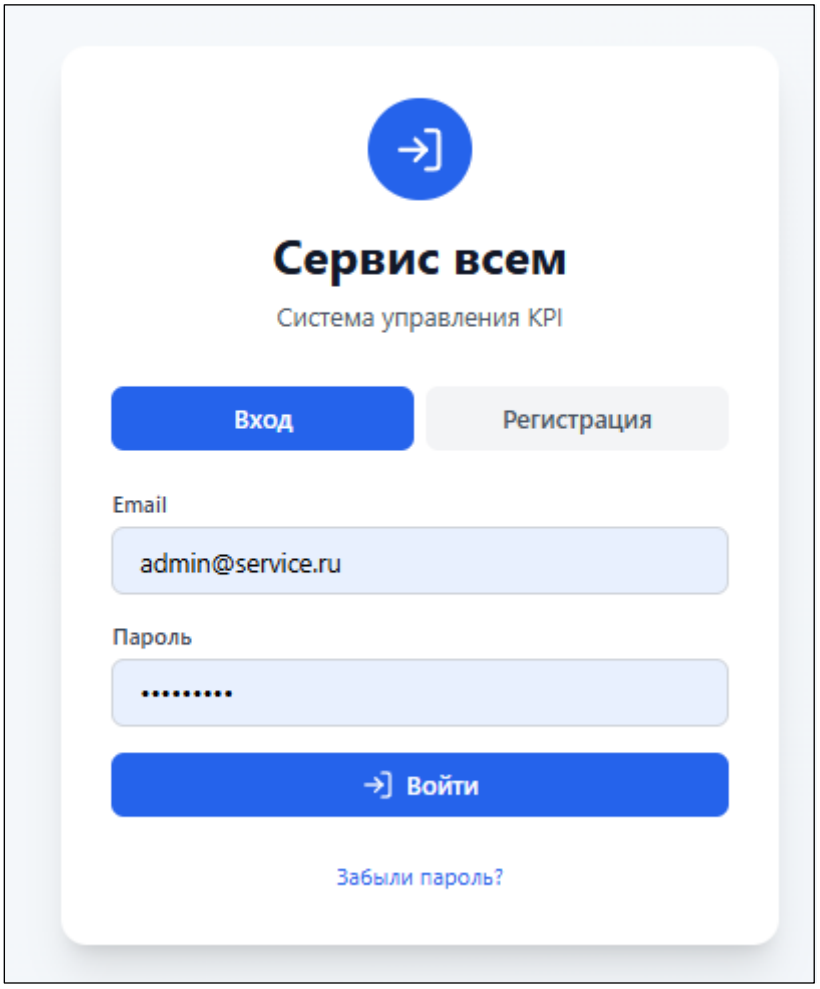

Рисунок 2.19 – Страница резервного копирования

На странице, наблюдаемой на рисунке 2.19 показано окно авторизации (входа).

2.4 Инструкция по работе с продуктом

В данном разделе будут указаны основные функции, реализованные в дашборде по указанным требованиям, подробное описание всего функционала и работы с ним:

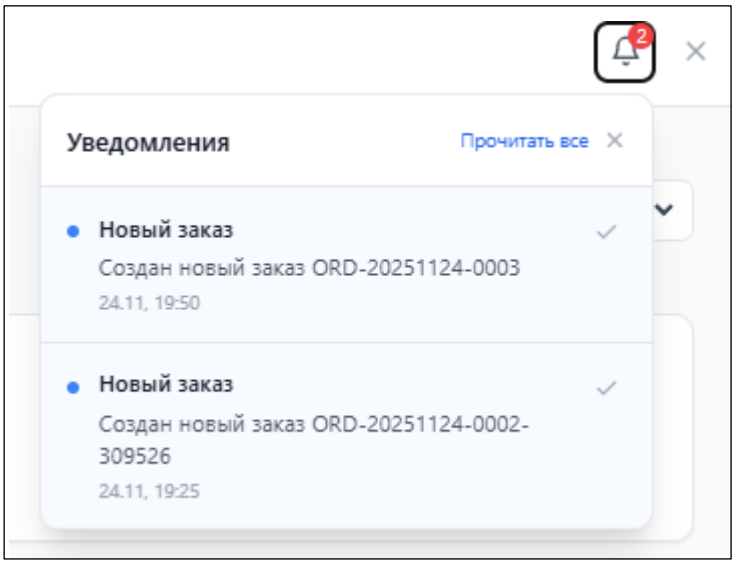

Рисунок 2.20 – Уведомления

На рисунке 2.20 можно наблюдать уведомления, оповещающие когда появляется новый заказ или нанимается новый сотрудник.

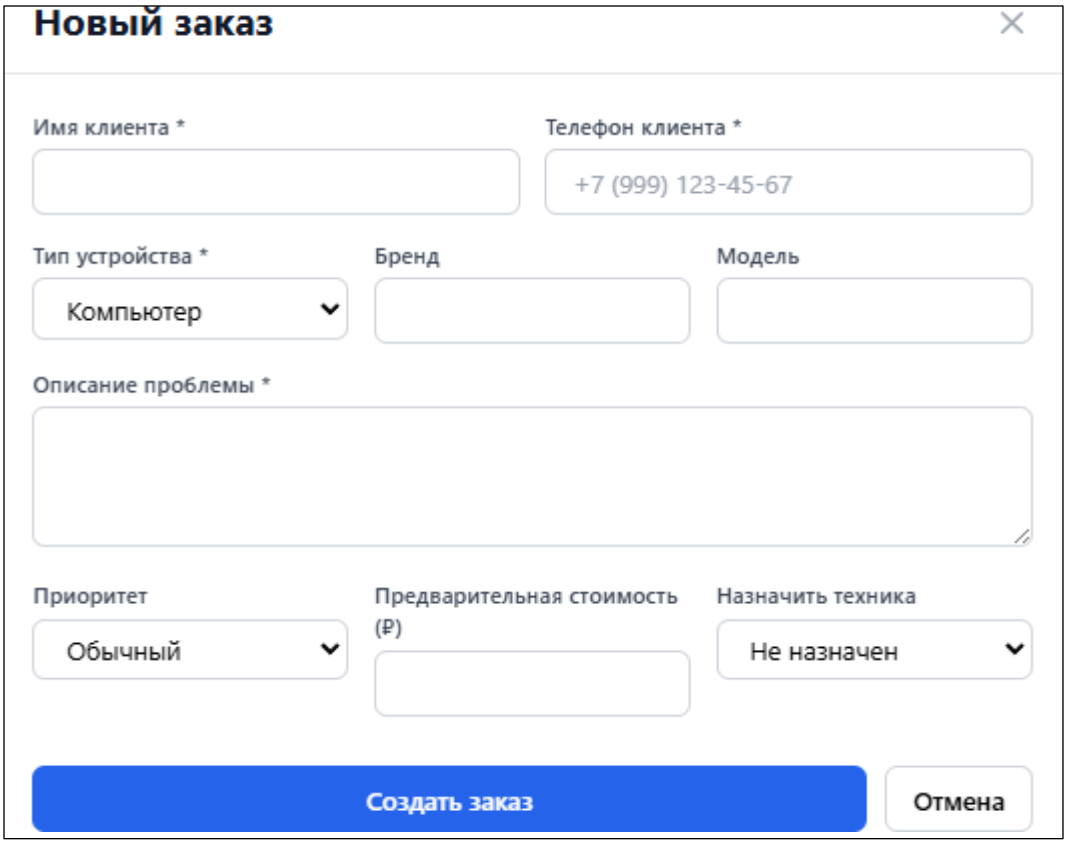

Рисунок 2.21 – Окно создания заказа

На рисунке 2.21 можно наблюдать окно создания нового заказа, где мы заполняем контактные и аппаратные данные, а также назначаем техника кто будет выполнять этот заказ и выставляем приоритетность заказа, а также можешь прикреплять файлы, такие как фотографии, чеки, текст и т.д.

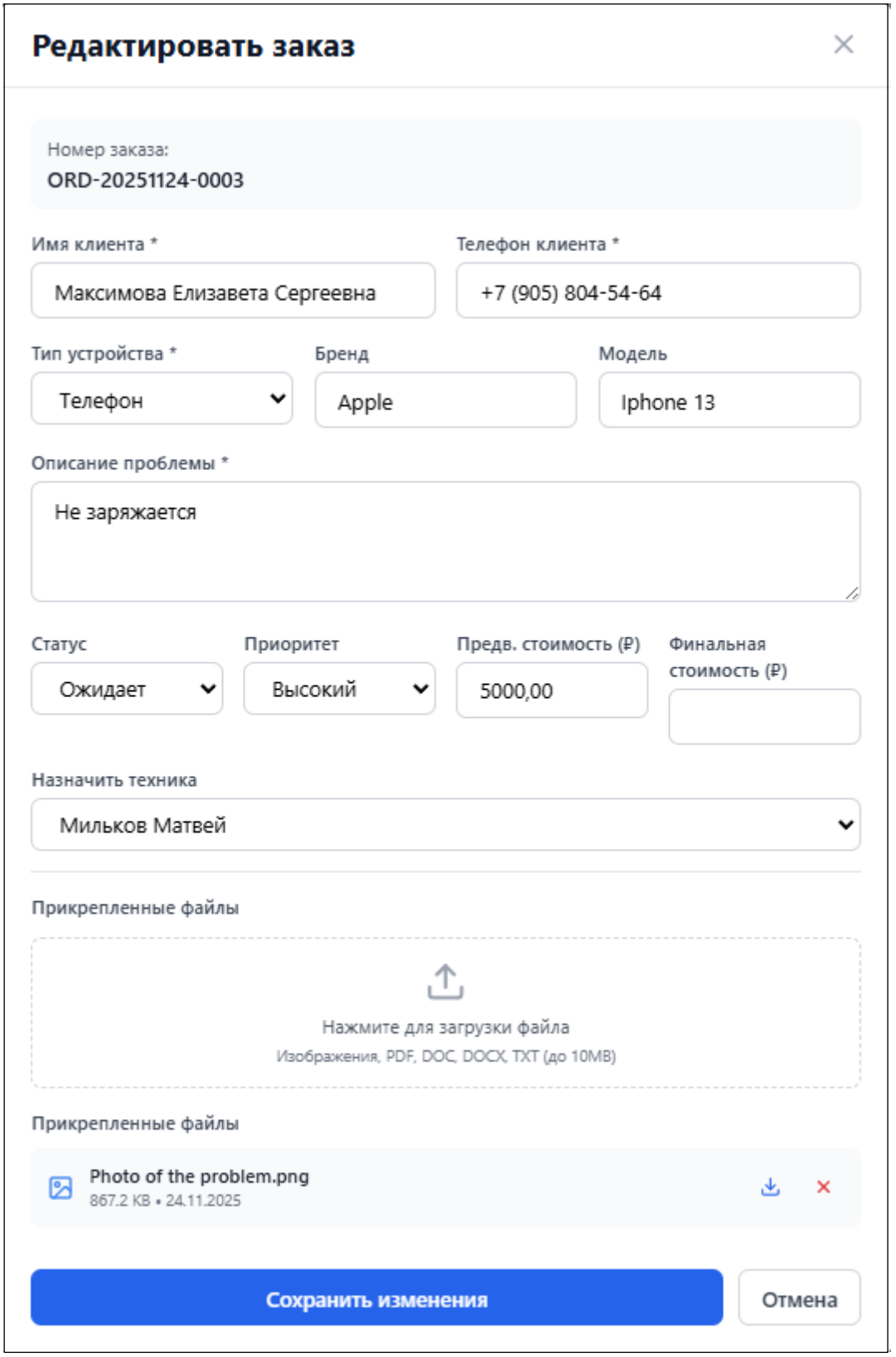

Рисунок 2.22 – Окно редактирования заказа

На рисунке 2.22 можно наблюдать окно редактирования существующего заказа, а также прикрепить новые фотографии или техническую сводку старого специалиста.

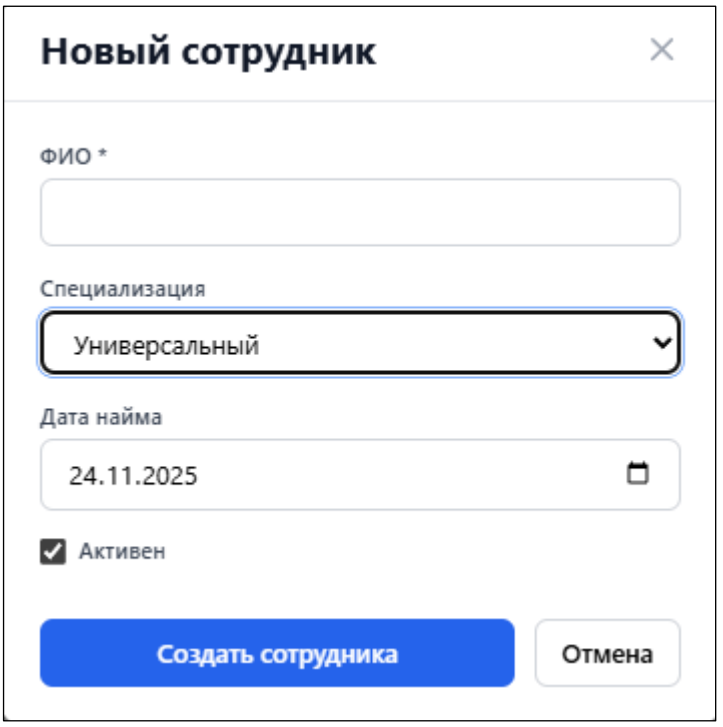

Рисунок 2.23 – Окно добавления сотрудника

На рисунке 2.23 можно наблюдать окно добавления (найма) сотрудника, где указываем ФИО, специализацию и дату найма сотрудника, а также активен он в данный момент или нет.

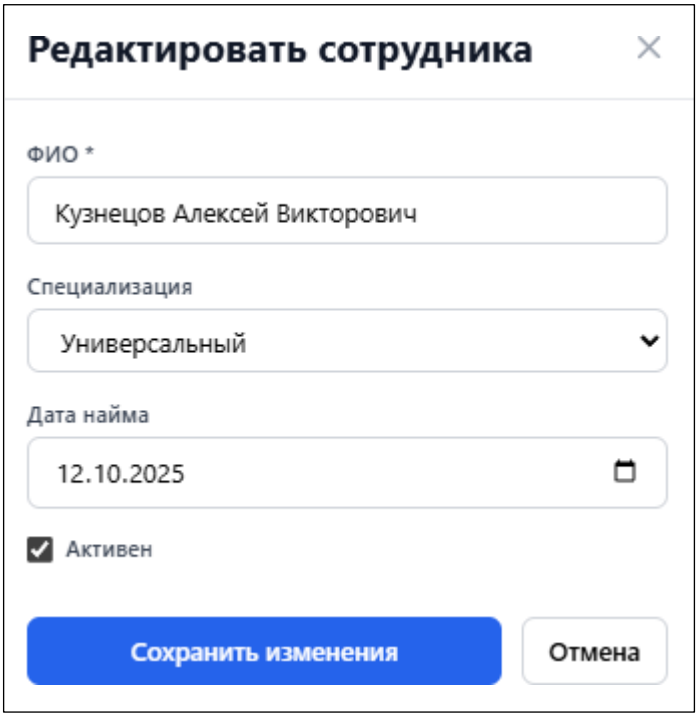

Рисунок 2.24 – Окно редактирования сотрудника

На рисунке 2.24 можно наблюдать окно редактирования сотрудника, здесь можно исправить ФИО в случае его смены или опечатки, изменить специализацию, если сотрудник переквалифицировался или был назначен на новую специализацию

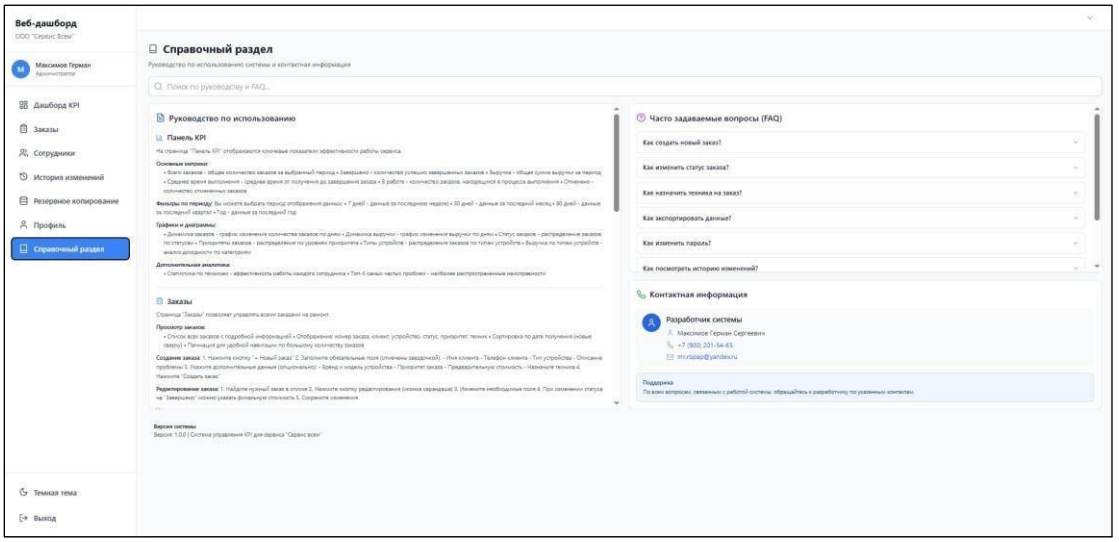

Рисунок 2.25 – Страница справочного раздела

На рисунке 2.25 можно наблюдать справочный раздел, в котором написано руководство пользователя, часто задаваемые вопросы, а также как работать с заказами, сотрудниками, аудитом, профилем и резервным копированием.

Листинг основного функционала можно увидеть в приложении А.

Заключение

Подводя итоги, можно сказать, что поставленные цели и задачи выполнены, приложение корректно работает и готово к использованию или дальнейшей доработке. Разработанный дашборд позволит предприятию «Сервис всем» автоматизировать процессы сбора, расчёта и визуализации KPI, которые ранее выполнялись вручную. Это ускорит формирование аналитической отчётности, позволит руководству и сотрудникам уделять больше времени стратегическому планированию и оперативному управлению, а также снизит нагрузку на отделы за счёт исключения рутинных операций.

Поставленная тема является многогранной и практически значимой, поскольку потребовала применения современных технологий веб-разработки, работы с базами данных и создания сложной логики расчёта метрик. В процессе работы были изучены и применены такие технологии, как React, TypeScript, Node.js, Express и PostgreSQL, углублены знания в области визуализации данных и проектирования пользовательских интерфейсов. Дипломный проект можно считать успешно выполненным и полностью соответствующим поставленным требованиям.

Библиография

1 React: Документация и руководства [Электронный ресурс]. – Режим доступа: https://react.dev/ (дата обращения: 24.11.2025).

2 TypeScript: Официальная документация [Электронный ресурс]. – Режим доступа: https://www.typescriptlang.org/docs/ (дата обращения: 25.11.2025)

3 Node.js: Руководство для разработчиков [Электронный ресурс]. – Режим доступа: https://nodejs.org/en/docs/ (дата обращения: 26.11.2025).

4 Express.js: Документация фреймворка [Электронный ресурс]. – Режим доступа: https://expressjs.com/ (дата обращения: 26.11.2025).

5 PostgreSQL: Официальная документация [Электронный ресурс]. – Режим доступа: https://www.postgresql.org/docs/ (дата обращения: 27.11.2025).

6 Chart.js: Руководство по визуализации данных [Электронный ресурс]. – Режим доступа: https://www.chartjs.org/docs/latest/ (дата обращения: 28.11.2025).

7 Tailwind CSS: Документация по стилизации [Электронный ресурс]. – Режим доступа: https://tailwindcss.com/docs (дата обращения: 28.11.2025).

8 Vite: Руководство по сборке проектов [Электронный ресурс]. – Режим доступа: https://vitejs.dev/guide/ (дата обращения: 28.11.2025).

Приложение А
(справочное)
Листинг программы

Листинг Server.js:

```javascript
import express from 'express';
import cors from 'cors';
import dotenv from 'dotenv';
import authRoutes from './routes/auth.js';
import dataRoutes from './routes/data.js';
import pool from './db.js';

dotenv.config();

const app = express();
const PORT = process.env.PORT || 3001;

// Middleware
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Health check
app.get('/health', async (req, res) => {
  try {
    await pool.query('SELECT 1');
    res.json({ status: 'ok', database: 'connected' });
  } catch (error) {
        res.status(500).json({ status: 'error', database:
'disconnected', error: error.message });
  }
});

// Routes
app.use('/api/auth', authRoutes);
app.use('/api/data', dataRoutes);

// Initialize database with test user if needed
async function initializeDatabase() {
  try {
    const adminCheck = await pool.query(
      "SELECT id FROM profiles WHERE email = 'admin@service.ru'"
    );

    if (adminCheck.rows.length === 0) {
      console.log('Creating test admin user...');
        const userIdResult = await pool.query('SELECT
gen_random_uuid() as id');
      const userId = userIdResult.rows[0].id;
```

```
        await pool.query(
          `INSERT INTO profiles (id, email, full_name, role)
           VALUES ($1, $2, $3, $4)`,
          [userId, 'admin@service.ru', 'Администратор', 'admin']
        );
         console.log('Test admin user created: admin@service.ru /
admin123');
      } else {
        console.log('Test admin user already exists: admin@service.ru
/ admin123');
      }
    } catch (error) {
      console.error('Database initialization error:', error);
    }
  }

  // Start server
  app.listen(PORT, async () => {
    console.log(`Server running on http://localhost:${PORT}`);
    await initializeDatabase();
  });
  ```
```

Листинг Auth.js:

```javascript
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import pool from './db.js';

const JWT_SECRET = process.env.JWT_SECRET ||
'your_super_secret_jwt_key_change_in_production';

// Хеширование пароля
export async function hashPassword(password) {
  return bcrypt.hash(password, 10);
}

// Сравнение пароля с хешем
export async function comparePassword(password, hash) {
  return bcrypt.compare(password, hash);
}

// Генерация JWT токена (срок действия 30 дней)
export function generateToken(userId, role) {
  return jwt.sign({ userId, role }, JWT_SECRET, { expiresIn: '30d'
});
}

// Верификация JWT токена
export function verifyToken(token) {
  try {
    return jwt.verify(token, JWT_SECRET);
  } catch (error) {
    return null;
```
```

```
    }
  }

  // Middleware для проверки аутентификации
  export async function authenticateUser(req, res, next) {
    try {
      const authHeader = req.headers.authorization;
      if (!authHeader || !authHeader.startsWith('Bearer ')) {
        return res.status(401).json({ error: 'Unauthorized' });
      }

      const token = authHeader.substring(7);
      const decoded = verifyToken(token);

      if (!decoded) {
        return res.status(401).json({ error: 'Invalid token' });
      }

      const result = await pool.query(
        'SELECT id, email, full_name, role FROM profiles WHERE id =
$1',
        [decoded.userId]
      );

      if (result.rows.length === 0) {
        return res.status(401).json({ error: 'User not found' });
      }

      req.user = result.rows[0];
      next();
    } catch (error) {
      console.error('Auth error:', error);
      res.status(401).json({ error: 'Unauthorized' });
    }
  }
  ```
```

Листинг Data.js:

```
  ```javascript
  import express from 'express';
  import pool from '../db.js';
  import { authenticateUser } from '../auth.js';

  const router = express.Router();

  // Все роуты требуют аутентификации
  router.use(authenticateUser);

  // Создание заказа на обслуживание
  router.post('/orders', async (req, res) => {
    try {
      const {
        customer_name,
        customer_phone,
        device_type,
```

```
          device_brand,
          device_model,
          issue_description,
          status,
          priority,
          estimated_cost,
          assigned_to
        } = req.body;

         if (!customer_name || !customer_phone || !device_type ||
!issue_description) {
            return res.status(400).json({ error: 'Required fields:
customer_name, customer_phone, device_type, issue_description' });
        }

        // Генерация уникального номера заказа
       const today = new Date().toISOString().split('T')[0].replace(/-
/g, '');
        const orderNumberResult = await pool.query(
             "SELECT COUNT(*) as count FROM service_orders WHERE
order_number LIKE $1",
          [`ORD-${today}-%`]
        );
        let count = parseInt(orderNumberResult.rows[0].count) + 1;
         let orderNumber = `ORD-${today}-${String(count).padStart(4,
'0')}`;

        // Создание заказа
        const result = await pool.query(
          `INSERT INTO service_orders (
            order_number, customer_name, customer_phone, device_type,
device_brand,
                device_model, issue_description, status, priority,
estimated_cost, assigned_to
          )
          VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11)
          RETURNING *`,
          [
            orderNumber,
            customer_name,
            customer_phone,
            device_type,
            device_brand || null,
            device_model || null,
            issue_description,
            status || 'pending',
            priority || 'normal',
            estimated_cost || null,
            assigned_to || null
          ]
        );

        const newOrder = result.rows[0];
```

```
      // Создание записи в журнале аудита
      try {
        if (req.user && req.user.id) {
          await pool.query(
            `INSERT INTO audit_log (entity_type, entity_id, action,
changed_by, old_values, new_values)
             VALUES ($1, $2, $3, $4, $5, $6)`,
            [
              'order',
              newOrder.id,
              'create',
              req.user.id,
              null,
              JSON.stringify(newOrder)
            ]
          );
        }
      } catch (auditError) {
        console.error('Error creating audit log:', auditError);
      }

      res.status(201).json(newOrder);
    } catch (error) {
      console.error('Error creating order:', error);
      res.status(500).json({ error: 'Internal server error' });
    }
  });

  // Обновление заказа с отслеживанием изменений
  router.put('/orders/:id', async (req, res) => {
    try {
      const { id } = req.params;
      const updateData = req.body;

      // Получение старых значений для журнала аудита
      const oldResult = await pool.query(
        `SELECT so.*, t.full_name as technician_name
         FROM service_orders so
         LEFT JOIN technicians t ON so.assigned_to = t.id
         WHERE so.id = $1`,
        [id]
      );

      if (oldResult.rows.length === 0) {
        return res.status(404).json({ error: 'Order not found' });
      }

      const oldValues = oldResult.rows[0];

      // Обновление заказа
      const result = await pool.query(
        `UPDATE service_orders SET
          customer_name = COALESCE($1, customer_name),
          customer_phone = COALESCE($2, customer_phone),
```

```
          device_type = COALESCE($3, device_type),
          status = COALESCE($4, status),
          priority = COALESCE($5, priority),
          assigned_to = COALESCE($6, assigned_to),
          updated_at = now()
        WHERE id = $7
        RETURNING *`,
        [
               updateData.customer_name,  updateData.customer_phone,
updateData.device_type,
                          updateData.status,    updateData.priority,
updateData.assigned_to,
          id
        ]
      );

      // Получение новых значений с именем техника
      const newResult = await pool.query(
        `SELECT so.*, t.full_name as technician_name
         FROM service_orders so
         LEFT JOIN technicians t ON so.assigned_to = t.id
         WHERE so.id = $1`,
        [id]
      );

      const newValues = newResult.rows[0];

      // Создание записи в журнале аудита
      try {
        await pool.query(
            `INSERT INTO audit_log (entity_type, entity_id, action,
changed_by, old_values, new_values)
           VALUES ($1, $2, $3, $4, $5, $6)`,
          [
            'order',
            id,
            'update',
            req.user.id,
            JSON.stringify(oldValues),
            JSON.stringify(newValues)
          ]
        );
      } catch (auditError) {
        console.error('Error creating audit log:', auditError);
      }

      res.json(newValues);
    } catch (error) {
      console.error('Error updating order:', error);
      res.status(500).json({ error: 'Internal server error' });
    }
  });

  // Создание резервной копии
```

```
router.post('/backup/create', async (req, res) => {
  try {
    // Получение всех данных из базы
      const ordersResult = await pool.query('SELECT * FROM
service_orders ORDER BY received_date DESC');
      const techniciansResult = await pool.query('SELECT * FROM
technicians ORDER BY hire_date DESC');
      const usersResult = await pool.query('SELECT id, email,
full_name, role, created_at FROM profiles ORDER BY created_at DESC');
  const auditLogResult = await pool.query('SELECT * FROM audit_log
ORDER BY created_at DESC');
      const notificationsResult = await pool.query('SELECT * FROM
notifications ORDER BY created_at DESC');

    const backup = {
      version: '1.0',
      created_at: new Date().toISOString(),
      data: {
        orders: ordersResult.rows,
        technicians: techniciansResult.rows,
        users: usersResult.rows,
        audit_log: auditLogResult.rows,
        notifications: notificationsResult.rows
      }
    };

    res.json(backup);
  } catch (error) {
    console.error('Error creating backup:', error);
    res.status(500).json({ error: 'Internal server error' });
  }
});

// Восстановление из резервной копии
router.post('/backup/restore', async (req, res) => {
  try {
    const { backup } = req.body;

    if (!backup || !backup.data) {
      return res.status(400).json({ error: 'Invalid backup format'
});
    }

    const client = await pool.connect();
    try {
      await client.query('BEGIN');

      // Очистка существующих данных (кроме пользователей)
      await client.query('DELETE FROM notifications');
      await client.query('DELETE FROM audit_log');
      await client.query('DELETE FROM service_orders');
      await client.query('DELETE FROM technicians');

      // Восстановление заказов
```

```
        if (backup.data.orders && backup.data.orders.length > 0) {
          for (const order of backup.data.orders) {
            await client.query(
              `INSERT INTO service_orders (
                  id, order_number, customer_name, customer_phone,
device_type,
               device_brand, device_model, issue_description, status,
priority,
                    received_date, completed_date, estimated_cost,
final_cost, assigned_to
              ) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11,
$12, $13, $14, $15)
              ON CONFLICT (id) DO NOTHING`,
              [
                 order.id, order.order_number, order.customer_name,
order.customer_phone,
                            order.device_type,   order.device_brand,
order.device_model, order.issue_description,
                 order.status, order.priority, order.received_date,
order.completed_date,
                           order.estimated_cost,   order.final_cost,
order.assigned_to
              ]
            );
          }
        }

        // Восстановление техников
       if (backup.data.technicians && backup.data.technicians.length
> 0) {
          for (const tech of backup.data.technicians) {
            await client.query(
              `INSERT INTO technicians (
                id, full_name, specialization, hire_date, is_active
              ) VALUES ($1, $2, $3, $4, $5)
              ON CONFLICT (id) DO NOTHING`,
                    [tech.id, tech.full_name, tech.specialization,
tech.hire_date, tech.is_active]
            );
          }
        }

        await client.query('COMMIT');
        res.json({ message: 'Backup restored successfully' });
      } catch (error) {
        await client.query('ROLLBACK');
        throw error;
      } finally {
        client.release();
      }
    } catch (error) {
      console.error('Error restoring backup:', error);
      res.status(500).json({ error: 'Internal server error' });
    }
```

```
});
```

Листинг App.tsx:

```typescript
import { AuthProvider, useAuth } from './contexts/AuthContext';
import { ThemeProvider } from './contexts/ThemeContext';
import { LoginPage } from './components/LoginPage';
import { Dashboard } from './components/Dashboard';

function AppContent() {
  const { user, loading } = useAuth();

  if (loading) {
    return (
      <div className="min-h-screen flex items-center justify-center
bg-white dark:bg-gray-900">
        <div className="w-12 h-12 border-4 border-blue-600 border-
t-transparent rounded-full animate-spin" />
      </div>
    );
  }

  return user ? <Dashboard /> : <LoginPage />;
}

function App() {
  return (
    <ThemeProvider>
      <AuthProvider>
        <AppContent />
      </AuthProvider>
    </ThemeProvider>
  );
}

export default App;
```

Листинг api.ts:

```typescript
const API_URL = import.meta.env.VITE_API_URL ||
'http://localhost:3001/api';

class ApiClient {
  private getToken(): string | null {
    return localStorage.getItem('token');
  }

  private async request<T>(
    endpoint: string,
    options: RequestInit = {}
  ): Promise<T> {
    const token = this.getToken();
```

```
      const headers: HeadersInit = {
        'Content-Type': 'application/json',
        ...options.headers,
      };

      if (token) {
        headers['Authorization'] = `Bearer ${token}`;
      }

      const response = await fetch(`${API_URL}${endpoint}`, {
        ...options,
        headers,
      });

      if (!response.ok) {
         const error = await response.json().catch(() => ({ error:
'Network error' }));
           throw  new  Error(error.error  ||  `HTTP  error!  status:
${response.status}`);
      }

      return response.json();
    }

    async login(email: string, password: string) {
      const data = await this.request('/auth/login', {
        method: 'POST',
        body: JSON.stringify({ email, password }),
      });
      if (data.token) {
        localStorage.setItem('token', data.token);
      }
      return data;
    }

    async getOrders() {
      return this.request('/data/orders');
    }

    async createOrder(data: {
      customer_name: string;
      customer_phone: string;
      device_type: string;
      device_brand?: string;
      device_model?: string;
      issue_description: string;
      status?: string;
      priority?: string;
      estimated_cost?: number;
      assigned_to?: string;
    }) {
      return this.request('/data/orders', {
        method: 'POST',
        body: JSON.stringify(data),
```

```
      });
    }

    async updateOrder(id: string, data: {
      customer_name?: string;
      customer_phone?: string;
      device_type?: string;
      status?: string;
      priority?: string;
      assigned_to?: string;
    }) {
      return this.request(`/data/orders/${id}`, {
        method: 'PUT',
        body: JSON.stringify(data),
      });
    }

    async getAuditLog(entityType?: string, entityId?: string, limit?:
number) {
      const params = new URLSearchParams();
      if (entityType) params.append('entity_type', entityType);
      if (entityId) params.append('entity_id', entityId);
      if (limit) params.append('limit', limit.toString());
      const query = params.toString();
       return this.request(`/data/audit-log${query ? '?' + query :
''}`);
    }

    async createBackup() {
      return this.request('/data/backup/create', {
        method: 'POST',
      });
    }

    async restoreBackup(backup: any) {
      return this.request('/data/backup/restore', {
        method: 'POST',
        body: JSON.stringify({ backup }),
      });
    }

    logout() {
      localStorage.removeItem('token');
    }
  }

  export const api = new ApiClient();
  ```
```

Листинг DashboardPage.tsx:

```typescript
import { useEffect, useState, useCallback } from 'react';
import { api } from '../lib/api';
import { MetricCard } from '../components/MetricCard';
import { DonutChart } from '../components/charts/DonutChart';
```
```

```
import { LineChart } from '../components/charts/LineChart';
import { TrendingUp, CheckCircle, DollarSign, Clock } from 'lucide-
react';

export function DashboardPage() {
  const [orders, setOrders] = useState<ServiceOrder[]>([]);
  const [loading, setLoading] = useState(true);
  const [period, setPeriod] = useState<PeriodType>('7');

  const loadData = useCallback(async () => {
    try {
      setLoading(true);
      const ordersData = await api.getOrders();
      setOrders(ordersData || []);
    } catch (error) {
      console.error('Error loading data:', error);
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => {
    loadData();
  }, [loadData]);

  // Расчет метрик KPI
  const totalOrders = orders.length;
   const completedOrders = orders.filter(o => o.status ===
'completed').length;
   const cancelledOrders = orders.filter(o => o.status ===
'cancelled').length;
   const inProgressOrders = orders.filter(o => o.status ===
'in_progress').length;
   const pendingOrders = orders.filter(o => o.status ===
'pending').length;

  // Общая выручка
  const totalRevenue = orders
    .filter(o => o.status === 'completed')
      .reduce((sum, o) => sum + Number(o.final_cost ||
o.estimated_cost || 0), 0);

  // Среднее время выполнения
  const completedWithDate = orders.filter(o =>
    o.status === 'completed' && o.completed_date && o.received_date
  );
  const avgCompletionTime = completedWithDate.length > 0
    ? completedWithDate.reduce((sum, o) => {
        const start = new Date(o.received_date);
        const end = new Date(o.completed_date!);
         const hours = (end.getTime() - start.getTime()) / (1000 *
60 * 60);
        return sum + hours;
      }, 0) / completedWithDate.length
```

```
        : 0;

      // Распределение по статусам
      const orderStatusData = [
        { label: 'Завершено', value: completedOrders, color: '#10b981'
},
        { label: 'В работе', value: inProgressOrders, color: '#3b82f6'
},
        { label: 'Ожидает', value: pendingOrders, color: '#f59e0b' },
        { label: 'Отменено', value: cancelledOrders, color: '#ef4444'
},
      ].filter(item => item.value > 0);

      // Фильтрация заказов по периоду
      const getFilteredOrders = () => {
        const now = new Date();
        now.setHours(23, 59, 59, 999);
        const daysAgo = new Date();
        daysAgo.setHours(0, 0, 0, 0);

        switch (period) {
          case '7': daysAgo.setDate(now.getDate() - 7); break;
          case '30': daysAgo.setDate(now.getDate() - 30); break;
          case '90': daysAgo.setDate(now.getDate() - 90); break;
          case '365': daysAgo.setDate(now.getDate() - 365); break;
        }

        return orders.filter(order => {
          const orderDate = new Date(order.received_date);
          orderDate.setHours(0, 0, 0, 0);
          return orderDate >= daysAgo && orderDate <= now;
        });
      };

      const filteredOrders = getFilteredOrders();

      // Динамика заказов по дням
      const ordersByDate: Record<string, number> = {};
      filteredOrders.forEach(order => {
                          const date = new
Date(order.received_date).toLocaleDateString('ru-RU', { day: '2-
digit', month: '2-digit' });
        ordersByDate[date] = (ordersByDate[date] || 0) + 1;
      });

      const ordersData = Object.entries(ordersByDate)
        .sort(([a], [b]) => {
          const dateA = new Date(a.split('.').reverse().join('-'));
          const dateB = new Date(b.split('.').reverse().join('-'));
          return dateA.getTime() - dateB.getTime();
        })
        .map(([label, value]) => ({ label, value }));

      if (loading) {
```

```
        return (
          <div className="flex items-center justify-center h-full">
            <div className="w-12 h-12 border-4 border-blue-600 border-
t-transparent rounded-full animate-spin" />
          </div>
        );
      }

      return (
        <div className="p-6 space-y-6">
          {/* Карточки метрик */}
         <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-
4 gap-6">
            <MetricCard
              title="Всего заказов"
              value={totalOrders}
              icon={<TrendingUp />}
            />
            <MetricCard
              title="Завершено"
              value={completedOrders}
              icon={<CheckCircle />}
            />
            <MetricCard
              title="Выручка"
              value={`${totalRevenue.toLocaleString('ru-RU')} ₽`}
              icon={<DollarSign />}
            />
            <MetricCard
              title="Среднее время"
              value={`${Math.round(avgCompletionTime)} ч`}
              icon={<Clock />}
            />
          </div>

          {/* Графики */}
          <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
            <DonutChart data={orderStatusData} title="Распределение по
статусам" />
            <LineChart data={ordersData} title="Динамика заказов" />
          </div>
        </div>
      );
    }
    ```
```
```